Mô hình ngôn ngữ
Language model

Mô hình ngôn ngữ

- Là phân bố xác suất trên các tập văn bản
- Cho biết xác suất của 1 câu (hoặc 1 cụm từ) thuộc 1 ngôn ngữ là bao nhiêu
- Mô hình ngôn ngữ tốt sẽ đánh giá đúng các câu đúng ngữ pháp, trôi chảy hơn các từ có thứ tự ngẫu nhiên.
- Vd: P(“hôm nay trời đẹp”) > P(“trời đẹp nay hôm”)

Mô hình ngôn ngữ n-gram

- Mục tiêu: tính xác suất của 1 câu hoặc một cụm từ:

$$P(W) = P(w_1, w_2, w_3, w_4, w_5, \ldots, w_m)$$

- Theo công thức Bayes:

$$P(AB) = P(B|A) * P(A)$$

- Ta có:

$$P(w_1, w_2, w_3, w_4, w_5, \ldots, w_m) =$$
$$P(w_1)*P(w_2|w_1)*P(w_3|w_1w_2)*\ldots*P(w_m|w_1w_2w_3\ldots\ w_{m-1})$$

P(“hôm nay trời đẹp”) =

P(hôm)*P(nay|hôm)*P(trời|hôm nay)*P(đẹp |hôm nay trời)

Mô hình ngôn ngữ n-gram

- Cách tính xác suất:

$$\mathrm{P}(\text{đẹp} \mid hôm\ nay\ trời) = \frac{P(hôm\ nay\ trời\ đẹp)}{P(hôm\ nay\ trời)}$$

- không thể lưu hết các xác suất trên, đặc biệt với *m* là độ dài văn bản ngôn ngữ tự nhiên

→ sử dụng chuỗi Markov bậc *n* với giả thiết 1 từ chỉ phụ thuộc *n-1* từ đứng trước nó (mô hình n-gram)

$$\mathrm{P}(w_m|w_1w_2w_3\ldots w_{m-1}) = P(w_m|w_1, w_2, w_3, \ldots, w_{m-1})$$
$$= \mathrm{P}(w_m|w_{m-n+1}w_{m-n+2}\ldots w_{m-1})$$

Các mô hình n-gram

- Mô hình unigram (Xác suất của mỗi từ trong một chuỗi độc lập với xác suất của từ khác trong chuỗi):

$$P(w_1 w_2 \ldots wn) \sim \prod_i P(w_i)$$

- Mô hình bigram:

$$P(w_1 w_2 \ldots wn) \sim \prod_i P(w_i | wi_1)$$

- Mô hình trigram:

$$P(w_1 w_2 \ldots wn) \sim \prod_i P(w_i | wi_1 w_i_2)$$

Tính xác suất bigram

- Đánh giá Maximum Likelihood

$$P(w_i \mid w_{i-1}) = \frac{count(w_{i-1}, w_i)}{count(w_{i-1})}$$

$$P(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

Tính xác suất bigram

- Ví dụ 1:

  <s> I am Sam </s>

  <s> Sam I am </s>

  <s> I do not like green eggs and ham </s>

$$P(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

$P(\text{I} \mid \text{<s>}) = 2/3 = 0.67$ $P(\text{Sam} \mid \text{<s>}) = 1/3 = 0.33$

$P(\text{am} \mid \text{I}) = 2/3 = 0.67$ $P(\text{</s>} \mid \text{Sam}) = 1/2 = 0.5$

$P(\text{Sam} \mid \text{am}) = 1/2 = 0.5$ $P(\text{do} \mid \text{I}) = 1/3 = 0.33$

Tính xác suất bigram

❑ Ví dụ 2:

Berkeley Restaurant Project sentences

- can you tell me about any good cantonese restaurants close by
- mid priced thai food is what i’m looking for
- tell me about chez panisse
- can you give me a listing of the kinds of food that are available
- i’m looking for a good place to eat breakfast
- when is caffe venezia open during the day

Tính xác suất bigram

- Ví dụ 2:

  Đếm các bigram

  Trên tổng số 9222 câu

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Tính xác suất bigram

Ví dụ 2:

- Chuẩn hóa theo unigrams:

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

- Kết quả:

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Tính xác suất bigram

- Các xác suất đã tính được

P(<s> I want english food </s>) =

P(I|<s>)

× P(want|I)

× P(english|want)

× P(food|english)

× P(</s>|food)

= .000031

Các xác suất đã tính được

- Các xác suất đã tính được

P(english|want) = .0011

P(chinese|want) = .0065

P(to|want) = .66

P(eat | to) = .28

P(food | to) = 0

P(want | spend) = 0

P (i | <s>) = .25

Triển khai thực tế

- Sử dụng log thay cho phép nhân
  - Tránh được kết quả về 0
  - Nhanh hơn nhân

$$\log(p_1 \times p_2 \times p_3 \times p_4) = \log p_1 + \log p_2 + \log p_3 + \log p_4$$

Các mô hình ngôn ngữ có sẵn

- Google Book N-grams

  http://ngrams.googlelabs.com/

- KenLM

  https://kheafield.com/code/kenlm/

Đánh giá các mô hình ngôn ngữ

- Mô hình ngôn ngữ tốt sẽ gán xác suất cao cho các câu thực (đúng) hoặc các câu có tần suất xuất hiện lớn
  - Hơn các câu sai ngữ pháp hoặc các câu ít xuất hiện?
- Huấn luyện mô hình trên một tập huấn luyện (training set)
- Đánh giá trên một tập dữ liệu mới (test set)
- Sử dụng ma trận độ đo để đánh giá mức độ tốt của mô hình trên tập test

Đánh giá mô hình n-gram

- So sánh 2 mô hình A và B
  - Sử dụng mỗi mô hình cho một nhiệm vụ cụ thể:
    - sửa lỗi chính tả, nhận dạng tiếng nói, dịch máy, …
  - Thử nghiệm (chạy) nhiệm vụ đó, tính độ chính xác khi sử dụng mô hình A và B
    - Bao nhiêu từ sai được sửa đúng (sửa lỗi chính tả)
    - Bao nhiêu từ được dịch đúng (dịch máy)
  - So sánh độ chính xác khi sử dụng A và B

Đánh giá mô hình n-gram – Đánh giá trong

- Đánh giá trong sử dụng độ đo perplexity (độ phức tạp)
  - Đánh giá xấp xỉ không tốt
    - Chỉ khi dữ liệu test giống dữ liệu train (về bộ từ vựng)
    - Tốt cho thí nghiệm nhưng không tốt cho thực tế

Ý tưởng của Perplexity

- Shannon Game:
  - Ta có thể tiên đoán từ tiếp theo không?

    I always order pizza with cheese and _____

    The 33rd President of the US was _____

    I saw a _____

    mushrooms 0.1
    pepperoni 0.1
    anchovies 0.01
    ….
    fried rice 0.0001
    ….
    and 1e-100

  - Có thể dùng unigram không?
- Mô hình tốt sẽ gán xác suất cao cho từ thường xuyên xuất hiện ở vị trí dự đoán

Độ phức tạp (Perplexity)

- Mô hình tốt nhất là mô hình dự đoán từ chưa biết tốt nhất
  - Cho xác suất câu cao nhất P(W)

$$PP(W) = P(w_1 w_2 \ldots w_N)^{-\frac{1}{N}}$$

  - Perplexity là nghịch đảo xác suất trên tập test, chuẩn hóa theo số từ

$$= \sqrt[N]{\frac{1}{P(w_1 w_2 \ldots w_N)}}$$

- Luật chuỗi:

$$\mathrm{PP}(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i | w_1 \ldots w_{i-1})}}$$

- Với bigrams:

$$\mathrm{PP}(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i | w_{i-1})}}$$

Minimizing perplexity is the same as maximizing probability

Độ phức tạp (Perplexity)

- Độ phức tạp tương đương số trường hợp rẽ nhánh
- Giả thiết 1 câu gồm các chữ số ngẫu nhiên. Khi đó độ phức tạp của câu dựa trên 1 mô hình sẽ gán P=1/10

$$
\begin{aligned}
\mathrm{PP}(W) &= P(w_1 w_2 \ldots w_N)^{-\frac{1}{N}} \\
&= \left(\frac{1}{10}^{N}\right)^{-\frac{1}{N}} \\
&= \frac{1}{10}^{-1} \\
&= 10
\end{aligned}
$$

Hiện tượng quá khớp dữ liệu (overfitting)

- Mô hình n-grams chỉ tiên đoán từ tốt nếu tập test giống tập train.
  - Ta cần tạo ra mô hình có tính tổng quát, nghĩa là có thể xử lý các trường hợp xác suất = 0 (những TH không có trong tập train nhưng có trong tập test)

TH xác suất = 0

- Tập train:
  … denied the allegations
  … denied the reports
  … denied the claims
  … denied the request

- Tập test
  … denied the offer
  … denied the loan

P(“offer” | denied the) = 0

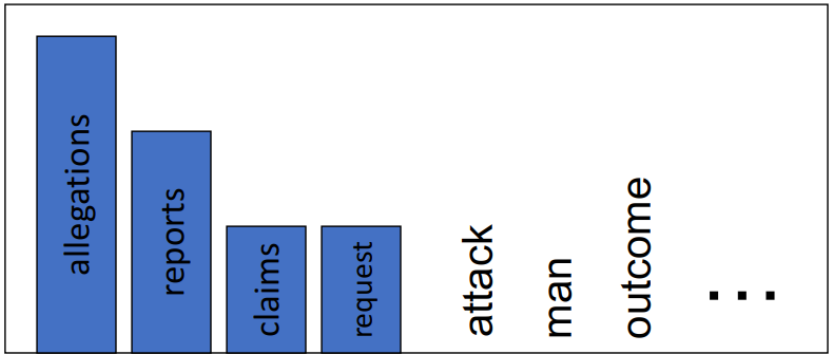
→ xác suất của 1 câu hoặc một cụm từ về 0

➢Sử dụng các phương pháp làm mịn

Ý tưởng của phương pháp làm mịn

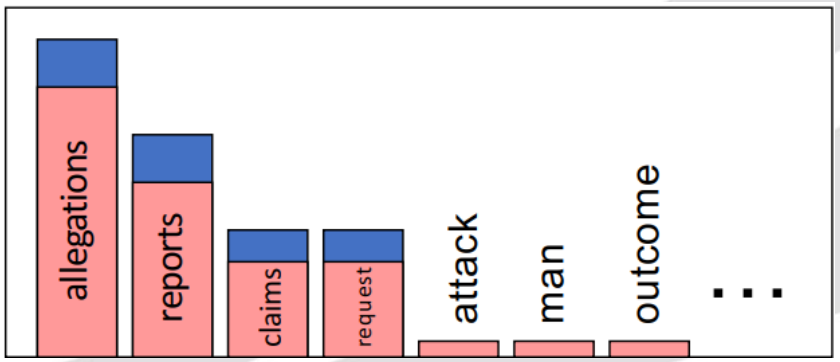
- Xác suất trên tập train:

P(w | denied the)
3 allegations
2 reports
1 claims
1 request
7 total



- Giảm xác suất các n-gram có xác suất lớn hơn 0 để bù cho các n-gram có xác suất bằng 0.

P(w | denied the)
2.5 allegations
1.5 reports
0.5 claims
0.5 request
2 other
7 total



Đánh giá kiểu add-one

- Gọi là phép làm mịn Laplace
- Giả thiết mỗi từ xuất hiện nhiều hơn 1 lần so với thực tế
- Cộng 1 vào tất cả các giá trị đếm

- Đánh giá MLE: $$P_{MLE}(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

- Đánh giá add-1: $$P_{Add-1}(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i)+1}{c(w_{i-1})+V}$$

V is the size of vocabulary

Đánh giá kiểu add-one

- Berkeley Restaurant Corpus : Laplace smoothed bigram counts

	i	want	to	eat	chinese	food	lunch	spend
i	6	828	1	10	1	1	1	3
want	3	1	609	2	7	7	6	2
to	3	1	5	687	3	1	7	212
eat	1	1	3	1	17	3	43	1
chinese	2	1	1	1	1	83	2	1
food	16	1	16	1	2	5	1	1
lunch	3	1	1	1	1	2	1	1
spend	2	1	2	1	1	1	1	1

Đánh giá kiểu add-one

- Add-1 là một công cụ thô sơ
  - Therefore, add-1 isn't used for n-grams:
  - We'll see better methods such as neural LM
- But add-1 is used to smooth other NLP models
  - For text classification
  - In domains where the number of zeros isn't so huge.
- Add-1 can be extended to add-k (k can be any positive real number, sometimes also called add-alpha)